

A Tutorial on CJR: Syntax, Semantics, and Separation Logic

Mechanized development in `theories/`

Abstract

We present CJR (Cangjie Radical), the performance-parity core of a Cangjie-like language: unboxed structs, word-addressed `CPointer` blocks, and final classes, together with `func`, `if`, and `while`. This note is a reading guide to the Coq/Iris mechanization. We give every concept twice—once as the definition that is actually checked, and once as ordinary mathematics with the same content—and we follow each program-logic rule by an account of what it is for. Finally, we walk through the examples that exercise the rules in order, and we conclude with a linked-list development, an array library, and an array list, each with the usual equations of that data structure.

1 What this development is

We take as our starting point a deliberately small core. CJR is sequential, monomorphic, and first-order; it has code pointers, so that a vtable can be written by hand. It has no generics, no inheritance, no `enum/match`, and no concurrency. Features of a larger source language that fall outside this cut are treated as sugar that must lower into the core at the same time and space cost. What follows defines that core. We do not define the lowering, a garbage collector, or byte-level layout.

Safety, in this development, is not a type system. The grammar `ty` records the types one intends to use, but the operational semantics is untyped. A program is safe when a weakest-precondition proof exists; adequacy then says that, from the empty machine, execution does not get stuck.

The Coq sources live under `theories/`. The language is an Iris evaluation-context language [6, 5], so the frame rule and the bind rule are Iris’s, rather than those of a separate Hoare calculus. The overall shape follows `HeapLang`, the example language shipped with the Iris Coq development [9].

2 Syntax

2.1 Concrete syntax

The programs in this note are written in Cangjie, as specified by the language documentation [1]. CJR is precisely the fragment of that language that the mechanization covers. The core in `theories/lang.v` is the lowering target, not a second surface syntax. A multi-argument `func` lowers to a single argument—a struct of the arguments—because core application is unary. A class that is not marked `open` cannot be inherited; that is the final class of the core. `CPointer` reads, writes, and offsets are the `unsafe` methods `read`, `write`, and `+` from the standard library, and allocation is the `foreign` function `malloc` from the Cangjie–C interoperability chapter.

```
// types used below: Int64, Float64, Bool, Unit, struct, class, CPointer<T>
```

```

func add(a: Int64, b: Int64): Int64 {
  a + b
}

let frozen: Int64 = 1
var n: Int64 = 1
while (1 <= n) {
  n = n - 1
}

if (n <= 0) {
  0
} else {
  n
}

struct Cell {
  public var x: Int64
  public init(x: Int64) { this.x = x }
}

class Counter {          // not open, so it has no subclasses
  public var value: Int64
  public init(value: Int64) { this.value = value }
}

foreign func malloc(size: UIntNative): CPointer<Unit>
foreign func free(ptr: CPointer<Unit>): Unit

```

`let` is immutable, and in the core it is substitution. `var` is mutable, and it is a stack slot. The last expression of a block is its value, and `()` is unit.

2.2 From Cangjie to the Coq core

We summarise the correspondence in the table below. Each row pairs one Cangjie form with the constructor in `theories/lang.v` to which it lowers. Field names become positions: the first field of a `struct` or `class` is index 0. A call `f(a, b)` is `App` of `f` to a `StructV` of the arguments, because `App` takes one argument. Forms that mention a string name of a `var` are rewritten by `subst_var` into the administrative stack operations listed in the last rows. Those operations have no spelling in Cangjie; they appear only after a `var` has been given a stack slot.

Cangjie	Coq constructor
Int64, Float64, Bool, Unit	TyInt, TyFloat, TyBool, TyUnit
struct S { ... }	TyStruct of the field types
CPointer<T>	TyPtr
class C { ... } with no open	TyObj of the field count
(T) -> U	TyFun
1, 1.0, true, ()	LitInt, LitFloat, LitBool, LitUnit
a CPointer value, including null from	LitPtr, LitNull
CPointer<T>()	LitObj
a class instance	Rec (some name, some parameter, body); a finished
func f(x) { e }	closure is RecV
a struct value such as Cell(0)	StructV
a name	Var
let x = e1 followed by e2	Let
var x = e1 followed by e2	VarBind
x = e for a var x	Assign
if (e0) { e1 } else { e2 }	If
while (e1) { e2 }	While
two expressions in sequence, or a block	Seq
+, -, <=, == on Int64, &&	PlusOp, MinusOp, LeOp, EqOp, AndOp
f(a)	App
Cell(e)	Struct, evaluated left to right
p.x on a struct	StructLoad at the index of x
p.x = e on a var struct	StructStore of that variable and index
Counter(e)	New
o.value	FieldLoad
o.value = e	FieldStore
malloc of one word, then CPointer<Int64>(...)	Alloc of 1, result LitPtr
free(p)	Free
unsafe { p.read() }	Load
unsafe { p.write(v) }	Store
unsafe { p + i }	Offset
no Cangjie spelling; introduced for a var slot	Pop, StackLoad, StackAssign, StackFieldStore, and the literal LitStack

The rest of this section defines those constructors and the machine on which they run.

2.3 Locations, identifiers, and types

A location is an integer. We wrap stack slots, raw words, and objects in distinct constructors so that the four ghost maps of the program logic cannot be confused.

```

Definition loc : Type := Z.
Inductive stack_id := StackId (z : Z).
Inductive raw_id := RawId (z : Z).
Inductive obj_id := ObjId (z : Z).

Inductive ty :=
  | TyInt | TyFloat | TyBool | TyUnit
  | TyStruct (ts : list ty)
  | TyPtr (t : ty)
  | TyObj (nfields : nat)
  | TyFun (t1 t2 : ty).

```

Informally, locations are $\mathbb{Z}$, and the three identifier sorts are the tagged copies $\text{StackId}(\ell)$, $\text{RawId}(\ell)$, and $\text{ObjId}(\ell)$. The type grammar is

$$\tau ::= \text{Int64} \mid \text{Float64} \mid \text{Bool} \mid \text{Unit} \mid \text{struct } \bar{\tau} \mid \text{CPointer } \tau \mid \text{class}_n \mid \tau \rightarrow \tau.$$

Here class_n means a final class with n fields. Nothing in the semantics consults τ . The type is documentation of the fragment; the theorems that follow do not assume it.

2.4 Values and expressions

A binder is an optional name: `None` is an ignored parameter. Literals cover integers, a float payload (for which no float operations are defined), booleans, unit, raw pointers, null, object references, and stack addresses. Values are literals, recursive closures, and unboxed struct tuples. Expressions comprise the core forms, together with administrative forms (`Pop`, `StackLoad`, `StackAssign`, `StackFieldStore`) that are introduced when a mutable variable is allocated.

```

Definition binder := option string.

Inductive base_lit :=
  | LitInt (n : Z) | LitFloat (n : Z) | LitBool (b : bool)
  | LitUnit | LitPtr (l : loc) | LitNull
  | LitObj (o : loc) | LitStack (l : loc).

Inductive val :=
  | LitV (l : base_lit)
  | RecV (f x : binder) (e : expr)
  | StructV (vs : list val)
with expr :=
  | Val (v : val) | Var (x : string)
  | Rec (f x : binder) (e : expr)
  | App (e1 e2 : expr)
  | Let (x : binder) (e1 e2 : expr)
  | VarBind (x : string) (e1 e2 : expr)
  | Assign (x : string) (e : expr)
  | Pop (l : stack_id) (e : expr)
  | StackLoad (e : expr)
  | StackAssign (e1 e2 : expr)
  | StackFieldStore (e1 : expr) (i : nat) (e2 : expr)
  | If (e0 e1 e2 : expr)
  | While (e1 e2 : expr)
  | Seq (e1 e2 : expr)
  | BinOp (op : bin_op) (e1 e2 : expr)
  | Alloc (e : expr) | Free (e : expr)
  | Load (e : expr) | Store (e1 e2 : expr)
  | Offset (e1 e2 : expr)
  | Struct (vs : list val) (es : list expr)
  | StructLoad (e : expr) (i : nat)
  | StructStore (x : string) (i : nat) (e : expr)
  | New (vs : list val) (es : list expr)
  | FieldLoad (e : expr) (i : nat)
  | FieldStore (e1 : expr) (i : nat) (e2 : expr).

```

In mathematics we write values

$$v ::= n \mid f \mid b \mid () \mid \text{ptr}(\ell) \mid \text{null} \mid \text{obj}(\ell) \mid \text{stk}(\ell) \mid \text{rec } f x. e \mid \langle v_0, \dots, v_{k-1} \rangle$$

and we use the expression forms by the same names as the constructors. `BinOp` ranges over `+`, `-`, `≤`, `=`, and boolean conjunction. Integer operations act on `LitInt`; conjunction acts on booleans. Anything else makes `bin_op_eval` return `None`, and the machine gets stuck.

Structs and objects are built by a list zipper. `Struct  $\bar{v} \bar{e}$` has already evaluated the values $\bar{v}$ and still has the expressions $\bar{e}$ to evaluate, left to right. `New` is the same zipper for a final class. When the expression list is empty, the zipper is done.

`Val` and `to_val` identify which expressions are finished:

```
Definition of_val (v : val) : expr := Val v.
Definition to_val (e : expr) : option val :=
  match e with Val v => Some v | _ => None end.
```

So the values of the language, as Iris sees them, are exactly the expressions `Val v`.

2.5 Two kinds of binding

We begin from a distinction that is already present in the surface language and then explain how the core encodes it. Immutable `let` and function parameters are substitution; mutable `var` is not. A `var` is given a fresh stack slot, and every use or assignment of that name is rewritten into an operation on the slot. A bare struct value is still copied: mutation in place happens only through a `var` slot or through a stored field.

Those surface facts come from the Cangjie 1.0 user manual [1]. The stack-slot encoding below is this note’s model of them, not a spelling in the language.

- **Immutable vs mutable bindings.** The chapter *Program Structure* (https://docs.cangjie-lang.cn/en/docs/1.0.0/user_manual/source_en/basic_programming_concepts/program_structure.html) introduces the mutability modifiers `let` and `var`: a `let` variable is assigned once (initialization), whereas a `var` variable may be reassigned. The chapter *Function Definition* (https://docs.cangjie-lang.cn/en/docs/1.0.0/user_manual/source_en/function/define_functions.html) states that function parameters are immutable and cannot be assigned to in the function body.
- **Struct copying.** The same *Program Structure* chapter distinguishes value types (including `struct`) from reference types (including `class`): assigning a value-type variable copies the instance; later mutation of one copy does not affect the other. *Creating a struct Instance* (https://docs.cangjie-lang.cn/en/docs/1.0.0/user_manual/source_en/struct/create_instance.html) repeats that assignment and parameter passing copy a `struct`, and that changing fields through an instance requires both the struct variable and the fields to be `var`. The related page *mut Function* (https://docs.cangjie-lang.cn/en/docs/1.0.0/user_manual/source_en/struct/mut.html) adds that `mut` methods need a mutable (`var`) receiver; they cannot be called on a `let` struct.
- **Stored class fields.** The chapter *Classes* (https://docs.cangjie-lang.cn/en/docs/1.0.0/user_manual/source_en/class_and_interface/class.html) states that `class` is a reference type: assignment does not copy the object, so field updates through one variable are visible through others that share it. That is the “stored field” case in the sentence above.

What follows—fresh stack slots, `subst_var`, `StackLoad/StackAssign/StackFieldStore`, `LitStack`, and `Pop`—is the Coq core’s administrative rewriting, not Cangjie surface syntax.

```
Definition subst_binder (b : binder) (v : val) (e : expr) : expr :=
  match b with None => e | Some x => subst x v e end.
```

The function `subst x v e` replaces free occurrences of x by the value v , and it does not descend under a binder that captures x . On a value, literals are unchanged and the body of a closure is substituted only when x is not the function name or the parameter.

The mutable variant `subst_var x l e` does a different job. A read `Var x` becomes `StackLoad(stk(l))`. An assignment `Assign x e` becomes `StackAssign(stk(l), e')`. A field write `StructStore x i e` becomes `StackFieldStore(stk(l), i, e')`. Other occurrences of x are rewritten recursively, again respecting capture. The slot address is itself a value, so later stack operations do not look up a string environment.

For a concrete example, the fragment `var x = 1 in (x = x + 1)` allocates a fresh slot ℓ , stores 1, and rewrites the body, reaching

$$\text{Pop}(\text{StackId}(\ell)) (\text{StackAssign}(\text{stk}(\ell), \text{BinOp} + (\text{StackLoad}(\text{stk}(\ell)), 1))).$$

Informally that is $\text{pop}_\ell(\text{stk}(\ell) := \text{stk}(\ell) + 1)$: the read of x is a load from the slot, the assignment writes the slot, and `Pop` deletes it when the body finishes.

What is striking about this split is that it is the whole model of “locals do not need a points-to fact until they are mutable.” An immutable binding disappears into the syntax. A mutable binding is a cell, and the cell has an owner.

3 Machine state

3.1 Three regions, one counter

The machine is a single record. Locals and unboxed structs that are stored in `var` slots live in the stack map. `CPointer` words live in the raw map, and each allocated block records its length in a second map on the same addresses. Final class fields live in a map from $(\text{ObjId}(\ell), i)$ to a value. A single counter `next` hands out fresh addresses for all three regions.

```
Record state := MkState {
  cjr_stack : gmap stack_id val;
  cjr_raw   : gmap raw_id val;
  cjr_blocks : gmap raw_id nat;
  cjr_obj   : gmap (obj_id * nat) val;
  cjr_next  : Z
}.
Definition state_init : state := inhabitant. (* all maps empty, next = 0 *)
```

Write a state as

$$\sigma = (\sigma_s, \sigma_r, \sigma_b, \sigma_o, \text{next}(\sigma)).$$

The initial state σ_0 is $(\emptyset, \emptyset, \emptyset, \emptyset, 0)$.

Indexing a pointer is arithmetic on the address, one word per element:

$$\text{ptr}(\ell) + i = \text{ptr}(\ell + i).$$

There is no byte offset and no padding. A block of length n at base ℓ occupies the words $\ell, \ell + 1, \dots, \ell + n - 1$. Blocks are freed explicitly. Class objects are not freed in this development: there is no `delete` for them, and there is no reachability argument.

Null is a pointer value, not an address in σ_r . Loading or storing through null has no rule, so it is stuck. An out-of-bounds or use-after-free access is stuck for the same reason: the lookup premise of load, store, or free fails.

3.2 Well-formed states

Not every 5-tuple is a state that the allocator is allowed to produce. Well-formedness says that every address already in use lies strictly below the counter, that every block's words are present, and that live blocks do not overlap.

```

Definition stack_below m n :=
  forall l, StackId l in dom m -> (l < n)%Z.
(* raw_below, block_below, obj_below are the same idea. *)

Definition block_cover blocks raw next :=
  forall l n, blocks !! RawId l = Some n ->
    (l + Z.of_nat n <= next)%Z /\
    forall i, i < n -> is_Some (raw !! RawId (l + Z.of_nat i)).

Definition blocks_disjoint blocks :=
  forall l1 n1 l2 n2,
    blocks !! RawId l1 = Some n1 ->
    blocks !! RawId l2 = Some n2 ->
    l1 <> l2 ->
    (l1 + Z.of_nat n1 <= l2 /\ l2 + Z.of_nat n2 <= l1)%Z.

Record state_wf (s : state) : Prop := {
  wf_stack : stack_below (cjr_stack s) (cjr_next s);
  wf_raw    : raw_below (cjr_raw s) (cjr_next s);
  wf_block  : block_below (cjr_blocks s) (cjr_next s);
  wf_obj    : obj_below (cjr_obj s) (cjr_next s);
  wf_cover  : block_cover (cjr_blocks s) (cjr_raw s) (cjr_next s);
  wf_disj   : blocks_disjoint (cjr_blocks s)
}.

```

In mathematics, $\text{wf}(\sigma)$ is the conjunction of:

$\forall \ell. \text{StackId}(\ell) \in \text{dom}(\sigma_s) \Rightarrow \ell < \text{next}(\sigma)$,
 and the same for raw addresses, block bases, and object addresses,
 $\forall \ell, n. \sigma_b(\text{RawId}(\ell)) = \text{Some}(n) \Rightarrow \ell + n \leq \text{next}(\sigma) \wedge \forall i < n. \sigma_r(\text{RawId}(\ell + i)) \neq \text{None}$,
 distinct live blocks have disjoint integer intervals.

The freshness lemmas are immediate from the first four clauses. If $\text{wf}(\sigma)$ and $\ell = \text{next}(\sigma)$, then ℓ is absent from the stack, from the raw heap, from the block map, and, at every field index, from the object heap. Allocation is therefore the insertion of a key that is not already present.

The initial state is well formed: `state_init_wf`. Each successful head step that the program logic uses is proved to preserve wf: stack allocation, deletion, and update; raw store; one-word allocation and freeing; class allocation; and field update.

One-word allocation is worth spelling out, because it is the case the logic rule talks about. From base $\ell = \text{next}(\sigma)$,

$$\begin{aligned}\sigma'_r &= \{\text{Rawld}(\ell) \mapsto 0\} \cup \sigma_r, \\ \sigma'_b &= \sigma_b[\text{Rawld}(\ell) \mapsto 1], \\ \text{next}' &= \ell + 1.\end{aligned}$$

The new word is in range, the new block is covered, and disjointness holds because the new interval $[\ell, \ell + 1)$ starts at the old counter, past every previous address. Freeing that block deletes the length and the word. Other blocks keep their cells precisely because their intervals were disjoint from $[\ell, \ell + 1)$.

Class allocation at $o = \text{next}(\sigma)$ inserts the fields $(\text{Objld}(o), i) \mapsto v_i$ for a list $v_0, \dots, v_{k-1}$ and advances the counter by one. Those keys are fresh because every old object address is below o .

4 Small-step semantics

4.1 Head reduction

A head step is a relation

$$(e, \sigma) \longrightarrow (e', \sigma')$$

with no observations and no forked threads. Both are hard-wired: **observation** is the empty type, and every rule produces an empty list of threads. The language is deterministic on the nose.

Pure computation. These rules neither read nor write the heaps.

$$\begin{aligned}\text{rec } f x. e &\longrightarrow \text{rec } f x. e && \text{(as a value)} \\ (\text{rec } f x. e) v &\longrightarrow e[x \mapsto v][f \mapsto \text{rec } f x. e] \\ \text{let } x = v \text{ in } e &\longrightarrow e[x \mapsto v] \\ \text{if true then } e_1 \text{ else } e_2 &\longrightarrow e_1 \\ \text{if false then } e_1 \text{ else } e_2 &\longrightarrow e_2 \\ \text{while } e_1 \text{ do } e_2 &\longrightarrow \text{if } e_1 \text{ then } (e_2; \text{while } e_1 \text{ do } e_2) \text{ else } () \\ v; e &\longrightarrow e \\ v_1 \oplus v_2 &\longrightarrow v && \text{when } \llbracket \oplus \rrbracket(v_1, v_2) = \text{Some}(v) \\ \text{ptr}(\ell) + i &\longrightarrow \text{ptr}(\ell + i) \\ \langle \bar{v}, v :: \bar{e} \rangle &\longrightarrow \langle \bar{v} ++ [v], \bar{e} \rangle \\ \langle \bar{v} \rangle &\longrightarrow \langle v_0, \dots \rangle && \text{(the struct value)} \\ \langle \bar{v} \rangle.i &\longrightarrow v && \text{when } v \text{ is the } i\text{th component.}\end{aligned}$$

The **while** rule duplicates the loop. It is a head step, not an evaluation context, so the copies are syntax rather than a shared mutable closure. The condition and the body are evaluated later, by ordinary contexts.

Stack. Let $\ell = \text{next}(\sigma)$.

$\text{var } x = v \text{ in } e \longrightarrow \text{pop}_\ell(e\{x \mapsto \ell\})$	and $\sigma_s(\text{StackId}(\ell)) := v$, $\text{next} := \ell + 1$
$\text{pop}_\ell v \longrightarrow v$	and delete the slot
$\text{stk}(\ell) \longrightarrow v$	if the slot holds v
$\text{stk}(\ell) := v \longrightarrow ()$	if the slot exists; it then holds v
$\text{stk}(\ell).i := v \longrightarrow ()$	if the slot holds a struct and index i is in range.

The last rule replaces component i with `set_nth` and writes the struct back into the same slot. No raw or object address is allocated. The most important consequence is that a stack update can be framed against a raw points-to fact: the two maps are different components of σ .

Raw heap. For $n > 0$, allocation of n words at $\ell = \text{next}(\sigma)$ returns $\text{ptr}(\ell)$, records length $Z.\text{to_nat}(n)$, zeroes the words, and sets the counter to $\ell + n$. The proved program-logic rule specialises this to $n = 1$.

$\text{free}(\text{ptr}(\ell)) \longrightarrow ()$	if a block length is recorded at ℓ ; that block's words are deleted
$!\text{ptr}(\ell) \longrightarrow v$	if the word holds v
$\text{ptr}(\ell) := v \longrightarrow ()$	if the word exists; it then holds v .

Free deletes whatever length is stored. The one-word reasoning rule asks separately for the length fact 1, so the client of that rule cannot free a longer block by mistake.

Objects. The object zipper steps like the struct zipper. When it finishes, at $o = \text{next}(\sigma)$,

$$\text{new}(v_0, \dots, v_{k-1}) \longrightarrow \text{obj}(o)$$

and the fields $(o, i) \mapsto v_i$ are added. Field load and field store are lookup and update at $(\text{ObjId}(o), i)$. Store requires the field to exist already. A missing field, like a missing word, is stuck.

4.2 Evaluation contexts

Head steps fire only at the root. The context items say where a subexpression may be reduced. Each item has one hole, and the hole is the expression currently being evaluated. Arguments are evaluated left to right. `if` evaluates only the condition. `while` does not evaluate under itself: the unfolding rule is the head step above.

Representative items, written as contexts:

$$\begin{aligned} &E[\text{let } x = [] \text{ in } e], E[\text{var } x = [] \text{ in } e], E[\text{pop}_\ell []], \\ &E[[] e], E[v []], E[\text{if } [] \text{ then } e_1 \text{ else } e_2], E[[]; e], \\ &E[[] \oplus e], E[v \oplus []], E[! []], E[\text{free } []], E[[] := e], E[v := []]. \end{aligned}$$

Struct and `new` items evaluate the next expression in the zipper. The Coq constructor list is `ectx_item`, and `fill_item` implements the grammar.

Iris then builds the language in the standard way.

```

Lemma cjr_ectxi_mixin : EctxiLanguageMixin of_val to_val fill_item base_step.
Canonical Structure cjr_ectxi_lang := EctxiLanguage cjr_ectxi_mixin.
Canonical Structure cjr_ectx_lang := EctxLanguageOfEctxi cjr_ectxi_lang.
Canonical Structure cjr_lang := LanguageOfEctx cjr_ectx_lang.

```

A configuration steps by finding a context K and a head redex:

$$K[e] \longrightarrow K[e'] \quad \text{when} \quad (e, \sigma) \longrightarrow (e', \sigma').$$

Filling a non-value never produces a value, a value never head-reduces, and a head reduction of a filled term implies that the hole was already a value. Those are the mixin obligations; they are what let Iris treat K as a program context in the bind rule.

4.3 Determinism

```
Lemma base_step_det e s k e' s' t k2 e2 s2 t2 :
  base_step e s k e' s' t ->
  base_step e s k2 e2 s2 t2 ->
  k = k2 /\ e' = e2 /\ s' = s2 /\ t = t2.
```

Theorem 1 (Determinism of head reduction). *If $(e, \sigma) \longrightarrow (e_1, \sigma_1)$ and $(e, \sigma) \longrightarrow (e_2, \sigma_2)$, then $e_1 = e_2$ and $\sigma_1 = \sigma_2$. Observations and forked threads, which are always empty, agree as well.*

The proof is inversion of both derivations. Where a rule reads a map or calls $\llbracket \oplus \rrbracket$, the two lookups are the same, so the two results are the same. There is no scheduler and no choice of fresh address other than $\text{next}(\sigma)$.

5 Separation logic

5.1 What is owned

The program logic does not talk about the whole machine. It talks about the cells a piece of code is allowed to use. Ownership is a ghost map—one authoritative map per region—with a points-to fact for each cell. Full ownership is the fraction 1.

```
Definition stack_mapsto (l : stack_id) (v : val) : iProp S :=
  ghost_map_elem stack_name l (DfracOwn 1) v.
Definition raw_mapsto (l : raw_id) (v : val) : iProp S :=
  ghost_map_elem raw_name l (DfracOwn 1) v.
Definition block_mapsto (l : raw_id) (n : nat) : iProp S :=
  ghost_map_elem block_name l (DfracOwn 1) n.
Definition obj_mapsto (o : obj_id) (i : nat) (v : val) : iProp S :=
  ghost_map_elem obj_name (o, i) (DfracOwn 1) v.
```

The notations in the sources are $\ell \mapsto_s v$, $\ell \mapsto_r v$, $\ell \mapsto_b n$, and $o \mapsto_o [i]v$. We use four maps, rather than a single heap, because a stack slot, a raw word, a block length, and a class field are different kinds of cell. A stack proof cannot accidentally be applied to a pointer.

The state interpretation ties the ghost maps to the concrete machine and remembers well-formedness:

```
Definition cjr_state_interp (s : state) : iProp S :=
  ghost_map_auth stack_name 1 (cjr_stack s) *
  ghost_map_auth raw_name 1 (cjr_raw s) *
  ghost_map_auth block_name 1 (cjr_blocks s) *
  ghost_map_auth obj_name 1 (cjr_obj s) *
  pure (state_wf s).
```

At every step, Iris owns $\text{StateInterp}(\sigma)$. A points-to fact is a fragment of one of those authoritative maps. Updating a cell is updating both the fragment and the matching entry of σ , and the wf conjunct is re-established by the lemma for that step.

5.2 Weakest precondition, in this instantiation

The Iris instance is the standard one for a deterministic language with no thread pool of interest:

$$\begin{aligned}\text{stateInterp}(\sigma) &= \text{cjr_state_interp}(\sigma), \\ \text{forkPost}(_) &= \top, \\ \text{numLaterPerStep}(_) &= 0.\end{aligned}$$

The weakest precondition $\text{wp } e \{ \Phi \}$ is Iris's. For this tutorial the reading is:

Definition 2 (Specification). *$\text{wp } e \{ \Phi \}$ means: for every well-formed extension of the current ghost ownership, executing e neither gets stuck nor forks, and if it terminates with a value v then $\Phi(v)$ holds. Partial correctness is enough: a diverging loop satisfies a weakest precondition whose postcondition is never reached.*

The later $\triangleright$ and the credit $\mathcal{L}1$ appearing in the rules are Iris's accounting for steps, in the mode `HasLc`. When a rule says

$$\ell \mapsto_r v \multimap (\mathcal{L}1 \multimap \ell \mapsto_r v \multimap \Phi(v)) \vdash \text{wp } (!\ell) \{ \Phi \},$$

the intended content is appealingly simple: ownership of the cell is enough to load it, and one gets the same cell back, together with the value that was there. The later and the credit are discharged by the step. In the example proofs they are thrown away with an introduction `iIntros " > _"`. They do not change the memory reasoning.

5.3 Structural rules

These three rules are why the logic is a separation logic rather than a Hoare logic with an explicit frame side-condition. Bind and the pure-step lemma are proved in `primitive_laws.v` by appealing to Iris. The frame rule is Iris's own `wp_frame_1`; this development does not re-prove it.

Lemma `wp_bind` $K \text{ e } \Phi$:
 $\text{wp } e \text{ (fun } v \Rightarrow \text{wp (fill } K \text{ (Val } v)) \Phi)} \vdash \text{wp (fill } K \text{ e) } \Phi$.

Lemma 3 (Bind). *If K is an evaluation context, then*

$$\text{wp } e \{ v. \text{wp } K[v] \{ \Phi \} \} \vdash \text{wp } K[e] \{ \Phi \}.$$

Intuition. To prove a specification of a term with a hole, we prove a specification of the hole, and we allow the postcondition of the hole to talk about the rest of the program. Left-to-right evaluation is just a choice of K . No memory fact has to be mentioned. Facts that the hole does not use stay outside this rule and are framed.

Lemma 4 (Frame).

$$P * \text{wp } e \{ \Phi \} \vdash \text{wp } e \{ v. P * \Phi(v) \}.$$

Intuition. If a proof of e never mentions P , then e cannot depend on the cells inside P , and those cells survive e unchanged. This is the rule that makes “the other pointer is untouched” a consequence of not putting that pointer into the callee’s precondition. It is not proved by induction on CJR expressions.

Lemma 5 (Consequence).

$$\text{wp } e \{ \Phi \} \wedge (\forall v. \Phi(v) \multimap \Psi(v)) \vdash \text{wp } e \{ \Psi \}.$$

Intuition. A stronger postcondition can be weakened. The examples use this when a lemma such as `bump_wp` returns both a pure result and a framed cell, and the continuation only needs one of them. In Coq it is `wp_wand` or `wp_mono`.

5.4 Pure rules

A pure head step is one that is defined in every state, does not change the state, and has exactly one successor. The generic lemma is `wp_pure_step`. Each concrete rule below is an instance. Where the step produces a value immediately, the postcondition is under a `later`, because that is how the generic lemma returns the credit. Where the step produces another expression, the rule simply asks for a weakest precondition of that expression.

Closures and application.

```

Lemma wp_rec f x e Phi :
  later (pound 1 -*- Phi (RecV f x e)) |- wp (Rec f x e) Phi.
Lemma wp_app f x e v Phi :
  wp (subst_binder x v (subst_binder f (RecV f x e) e)) Phi
  |- wp (App (Val (RecV f x e)) (Val v)) Phi.

```

$$\begin{aligned} & \triangleright(\Phi(\text{rec } f \ x. e)) \vdash \text{wp}(\text{rec } f \ x. e) \{ \Phi \}, \\ & \text{wp } e[x \mapsto v][f \mapsto \text{rec } f \ x. e] \{ \Phi \} \vdash \text{wp}((\text{rec } f \ x. e) v) \{ \Phi \}. \end{aligned}$$

Intuition. Evaluating a function literal yields the closure as a value. Calling it substitutes the argument and, if the function is recursive, the closure itself. No heap fact is required: the code is data, and the data is the value in hand. That is what lets a `vtable` be an ordinary struct of closures.

Let, if, while, sequencing, arithmetic, offset.

$$\begin{aligned} & \text{wp } e[x \mapsto v] \{ \Phi \} \vdash \text{wp}(\text{let } x = v \text{ in } e) \{ \Phi \} \\ & \text{wp } e_1 \{ \Phi \} \vdash \text{wp}(\text{if true then } e_1 \text{ else } e_2) \{ \Phi \} \\ & \text{wp } e_2 \{ \Phi \} \vdash \text{wp}(\text{if false then } e_1 \text{ else } e_2) \{ \Phi \} \\ & \text{wp}(\text{if } e_1 \text{ then } (e_2; \text{while } e_1 \text{ do } e_2) \text{ else } ()) \{ \Phi \} \vdash \text{wp}(\text{while } e_1 \text{ do } e_2) \{ \Phi \} \\ & \text{wp } e \{ \Phi \} \vdash \text{wp}(v; e) \{ \Phi \} \\ & \llbracket \oplus \rrbracket(v_1, v_2) = \text{Some}(v) \Rightarrow \triangleright(\Phi(v)) \vdash \text{wp}(v_1 \oplus v_2) \{ \Phi \} \\ & \triangleright(\Phi(\text{ptr}(\ell + i))) \vdash \text{wp}(\text{ptr}(\ell) + i) \{ \Phi \}. \end{aligned}$$

Intuition for while. The loop rule does not mention an invariant; it unfolds the loop once into a conditional. The invariant is whatever assertion one has in hand when returning to the same syntactic loop. If the condition is false, the unfolding takes the else branch and returns unit. If it is

true, the body runs and the same rule applies again. Termination is not proved; the unfolding is used only as many times as the concrete execution needs.

Intuition for offset. Adding an index to a pointer is arithmetic. It does not check that the resulting address lies in a block. The check happens when someone loads or stores: without a points-to fact at the computed address, the load rule does not apply, and the machine would be stuck. Bounds are therefore a matter of which points-to facts the proof owns, not of a side condition on $+$.

Struct values.

$$\begin{aligned} \triangleright(\Phi(\langle \bar{v} \rangle)) &\vdash \mathbf{wp} \langle \bar{v} \rangle \{ \Phi \}, \\ \bar{v}(i) = v &\Rightarrow \triangleright(\Phi(v)) \vdash \mathbf{wp} (\langle \bar{v} \rangle.i) \{ \Phi \}. \end{aligned}$$

Intuition. A finished zipper is the unboxed tuple. Reading a component copies it out. The tuple is not a heap cell, so there is nothing to frame and nothing to free. If the index is out of range, there is no rule.

Building a struct or an object from a non-empty expression list is also a pure step (**StructStepS**, **NewStepS**). The development uses **wp_pure_step** for that step in the vtable example rather than a named lemma.

5.5 Stack rules

```

Lemma wp_var_bind x v e Phi :
  (forall l, StackId l |->s v -*-
    wp (subst_var x l e)
      (fun w => exists v', StackId l |->s v' * Phi w))
  |- wp (VarBind x (Val v) e) Phi.

Lemma wp_stack_load l v Phi :
  l |->s v -*- later (pound 1 -*- l |->s v -*- Phi v)
  |- wp (StackLoad (stk l)) Phi.

Lemma wp_stack_assign l v w Phi :
  l |->s v -*- later (pound 1 -*- l |->s w -*- Phi unit)
  |- wp (StackAssign (stk l) (Val w)) Phi.

Lemma wp_stack_field l vs i w v Phi :
  vs !! i = Some w ->
  l |->s StructV vs -*-
  later (pound 1 -*- l |->s StructV (set_nth i v vs) -*- Phi unit)
  |- wp (StackFieldStore (stk l) i (Val v)) Phi.

```

In mathematics, with ℓ a stack address:

$$\begin{aligned} &(\forall \ell. \ell \mapsto_s v \text{ --* } \mathbf{wp} e \{ x \mapsto \ell \} \{ w. \exists v'. \ell \mapsto_s v' \text{ --* } \Phi(w) \}) \vdash \mathbf{wp} (\mathbf{var} \ x = v \text{ in } e) \{ \Phi \}, \\ &\ell \mapsto_s v \text{ --* } \triangleright(\ell \mapsto_s v \text{ --* } \Phi(v)) \vdash \mathbf{wp} (\mathbf{stk}(\ell)) \{ \Phi \}, \\ &\ell \mapsto_s v \text{ --* } \triangleright(\ell \mapsto_s w \text{ --* } \Phi(())) \vdash \mathbf{wp} (\mathbf{stk}(\ell) := w) \{ \Phi \}, \\ &\bar{v}(i) = w \Rightarrow \ell \mapsto_s \langle \bar{v} \rangle \text{ --* } \triangleright(\ell \mapsto_s \langle \bar{v}[i \mapsto u] \rangle \text{ --* } \Phi(())) \vdash \mathbf{wp} (\mathbf{stk}(\ell).i := u) \{ \Phi \}. \end{aligned}$$

Intuition for var. The rule allocates a fresh slot, hands exclusive ownership of it to the proof, and asks for a proof of the body after the name has been compiled into stack operations. The

contents may be changed. At the end the slot must be given back, at whatever value it then has. The **pop** administrative step deletes it, so the caller does not see a stack fact. This is lexical scope: the owner of the cell is the body, and the cell does not escape.

The existential is essential. If the postcondition demanded the original value, an assignment inside the body could not be verified, because assignment replaces the points-to fact.

Intuition for load, assign, and field update. Load reads and returns the same fact. Assignment exchanges the fact for one about the new value. Field update is assignment of a struct, with one component changed, and it is only available when that component already exists. A missing component is stuck, and there is no points-to fact that would let the rule fire. None of these rules allocates on the raw heap or the object heap.

5.6 Raw-heap rules

```

Lemma wp_alloc_one Phi :
  (forall l, RawId l |->r LitV (LitInt 0) *-
    RawId l |->b 1 *- Phi (LitV (LitPtr l)))
  |- wp (Alloc (Val (LitV (LitInt 1)))) Phi.

Lemma wp_free_one l v Phi :
  RawId l |->r v *- RawId l |->b 1 *- later (Phi unit)
  |- wp (Free (Val (LitV (LitPtr l)))) Phi.

Lemma wp_load l v Phi :
  RawId l |->r v *- later (RawId l |->r v *- Phi v)
  |- wp (Load (Val (LitV (LitPtr l)))) Phi.

Lemma wp_store l v w Phi :
  RawId l |->r v *- later (RawId l |->r w *- Phi unit)
  |- wp (Store (Val (LitV (LitPtr l))) (Val w)) Phi.

```

$$\begin{aligned}
& (\forall \ell. \ell \mapsto_r 0 * \ell \mapsto_b 1 * \Phi(\text{ptr}(\ell))) \vdash \text{wp}(\text{alloc}(1)) \{\Phi\}, \\
& \ell \mapsto_r v * \ell \mapsto_b 1 * \triangleright(\Phi(())) \vdash \text{wp}(\text{free}(\ell)) \{\Phi\}, \\
& \ell \mapsto_r v * \triangleright(\ell \mapsto_r v * \Phi(v)) \vdash \text{wp}(!\ell) \{\Phi\}, \\
& \ell \mapsto_r v * \triangleright(\ell \mapsto_r w * \Phi(())) \vdash \text{wp}(\ell := w) \{\Phi\}.
\end{aligned}$$

Intuition. Allocation creates a cell that was not owned before, initialised to zero, together with a separate fact that a block of length one starts there. The two facts have different jobs. The points-to fact lets one load and store the word. The block fact lets one free the block. Free consumes both, so the word cannot be used again: there is no fact left to feed to the load rule, and the concrete word has been deleted, so the machine would get stuck if one tried.

Load does not disturb the cell. Store keeps ownership and changes the value. Neither rule needs the block fact. A client may therefore load and store while the right to free sits unused in the context, and the frame rule keeps it there.

The operational rule for allocation accepts any positive length. The verified rule above is the one-word instance, which the small examples use. The array library uses the general rule: a points-to fact per word, plus one block fact for the base and the length.

5.7 Object rules

```

Lemma wp_new vs Phi :
  (forall o, ([* list] i |-> v in vs, ObjId o |->o[i] v) -*-
    Phi (LitV (LitObj o)))
  |- wp (New vs []) Phi.

Lemma wp_field_load o i v Phi :
  ObjId o |->o[i] v -*- later (ObjId o |->o[i] v -*- Phi v)
  |- wp (FieldLoad (Val (LitV (LitObj o))) i) Phi.

Lemma wp_field_store o i v w Phi :
  ObjId o |->o[i] v -*- later (ObjId o |->o[i] w -*- Phi unit)
  |- wp (FieldStore (Val (LitV (LitObj o))) i (Val w)) Phi.

```

$$\begin{aligned}
& (\forall o. \bigstar_{i < k} o \mapsto_o [i]v_i \multimap \Phi(\text{obj}(o))) \vdash \text{wp}(\text{new}(v_0, \dots, v_{k-1})) \{\Phi\}, \\
& o \mapsto_o [i]v \multimap \triangleright(o \mapsto_o [i]v \multimap \Phi(v)) \vdash \text{wp}(o.i) \{\Phi\}, \\
& o \mapsto_o [i]v \multimap \triangleright(o \mapsto_o [i]w \multimap \Phi(())) \vdash \text{wp}(o.i := w) \{\Phi\}.
\end{aligned}$$

Intuition. `new` of a finished argument list allocates a fresh object and returns one points-to fact per field, with the initialising values. There is no block fact and no free rule: the object stays allocated. Field load and field store are the same idea as raw load and store, on a different map. Owning $o \mapsto_o [i]v$ says nothing about $o \mapsto_o [j]w$ for $j \neq i$, and nothing about any raw word. That is why a raw cell can be framed across a field update, and why two fields of one object can be reasoned about separately.

5.8 Exclusivity

```

Lemma stack_mapsto_exclusive l v1 v2 :
  l |->s v1 -*- l |->s v2 |- False.
Lemma raw_mapsto_exclusive l v1 v2 :
  l |->r v1 -*- l |->r v2 |- False.

```

Lemma 6 (Exclusive ownership). *For a stack address or a raw address ℓ , and any values v_1, v_2 ,*

$$\ell \mapsto_s v_1 * \ell \mapsto_s v_2 \vdash \perp, \quad \ell \mapsto_r v_1 * \ell \mapsto_r v_2 \vdash \perp.$$

Intuition. Full ownership is unique. The ghost-map algebra rejects two fragments of fraction 1 at the same key, even if they agree on the value. This is the specification of non-aliasing. If a swap is specified with two points-to assumptions, applying it to one address twice is impossible: the precondition is false. The machine might still have a single cell there; the logic refuses to pretend there are two.

The same algebra applies to block facts and to object fields. The development records the stack and raw cases because those are the ones the examples need to point at.

6 Main theorems

6.1 Head reduction is deterministic

Theorem 1 above is `base_step_det`. Restated: the head reduction relation is a partial function of (e, σ) . Combined with the context machinery, a configuration has at most one successor. Stuckness is therefore unambiguous. Either the term is a value, or exactly one head redex is ready and the rule applies, or no rule applies and the program is stuck.

6.2 Well-formedness is an invariant of the verified steps

The state interpretation contains $\ulcorner \text{wf}(\sigma) \urcorner$. Each memory rule proves that the state after the step is well formed if the state before was. The statements are `wf_stack_alloc`, `wf_stack_delete`, `wf_stack_upd`, `wf_raw_store`, `wf_alloc_one`, `wf_free_one`, `wf_new`, and `wf_field_store`.

Lemma 7 (Allocator invariant). $\text{wf}(\sigma_0)$. *If $\text{wf}(\sigma)$ and a verified memory rule steps to σ' , then $\text{wf}(\sigma')$.*

The one-word cases are the ones used by the logic. For example, freeing a block of length 1 at ℓ preserves coverage of every other block because disjointness puts their words outside $\{\ell\}$.

6.3 Adequacy

```
Lemma cjr_adequacy S e (phi : val -> Prop) :
  (forall '(!cjrGS S) '(!invGS_gen HasLc S),
   |- wp e (fun v => pure (phi v))) ->
  adequate NotStuck e state_init (fun v _ => phi v).
```

Theorem 8 (Adequacy). *Let φ be a pure predicate on values. Suppose that in every Iris instantiation of the CJR ghost state one can prove*

$$\text{wp } e \{ v. \ulcorner \varphi(v) \urcorner \}.$$

Then, executing e from σ_0 , every reachable configuration can take a step or is a value, and every final value v satisfies $\varphi(v)$.

What the proof does. It allocates four empty ghost maps, packages them as the CJR ghost state, and observes $\text{wf}(\sigma_0)$. Iris’s adequacy theorem for weakest preconditions then applies. The hypothesis is a proof that does not depend on a particular ghost name, so it can be used at the names chosen here.

What it does not say. It does not say that every closed expression is safe. An expression with a free variable, a null dereference, or a load of an address nobody allocated has no such proof, and the theorem is silent. It also does not extract a functional-correctness result beyond the pure φ that was proved. If the postcondition is only “the result is unit”, adequacy says the program returns unit without getting stuck; the interesting memory facts were used and consumed inside the proof.

The closed examples `sum_one`, `bump_keep`, `cell`, `class_bump`, and `vtable_prog` are instantiated at this theorem. The other examples are open: their hypotheses are points-to facts, so they are stated as weakest preconditions rather than as runs from σ_0 .

7 The examples

Each example uses a feature that the previous one does not. They are the files in `theories/examples/`, in the order below. The programs are written in the Cangjie fragment above. The Coq definitions are the core `expr` terms those files actually prove.

7.1 Arithmetic: `arith.v`

Program.

```
// Generated by theories/examples/emit_cj.v from the CJR core terms.

foreign func malloc(size: UIntNative): CPointer<Unit>
foreign func free(ptr: CPointer<Unit>): Unit

func sumOne(): Int64 {
  var n = 1
  var s = 0
  while (1 <= n) {
    s = s + 1
    n = n - 1
  }
  s
}

main(): Unit {
  sumOne()
}
```

Theorem.

$$\text{wp sum_one} \{ v. \ulcorner v = 1 \urcorner \}.$$

From σ_0 , the program returns 1 and does not get stuck (`sum_one_adequate`).

What is new. Nothing is allocated on either heap. The proof uses `var`, stack load and assignment, `while`, `if`, sequencing, and integer `+` and `-`. The loop condition is true once: n is 1, then 0. On the true branch, s becomes 1 and n becomes 0. On the false branch the loop returns unit, and the trailing read returns s .

The stack facts at the end are $s \mapsto_s 1$ and $n \mapsto_s 0$. They are given back to `wp_var_bind` through the existential, and `pop` deletes them. The pure postcondition does not mention the stack. That is the scope rule from the previous section, exercised on a loop that really mutates.

7.2 Stack structs: `struct_upd.v`

Program.

```
// Generated by theories/examples/emit_cj.v from the CJR core terms.

foreign func malloc(size: UIntNative): CPointer<Unit>
foreign func free(ptr: CPointer<Unit>): Unit

struct Cell {
  public var x: Int64
}
```

```

    public init(x: Int64) {
        this.x = x
    }
}

func bump(): Int64 {
    var p = Cell(0)
    p.x = 1
    p.x
}

func bumpKeep(): Int64 {
    let r = unsafe { CPointer<Int64>(malloc(UIntNative(8 * 1))) }
    bump()
    unsafe { r.read() }
}

main(): Unit {
    bumpKeep()
}

```

Theorems.

$$\ell \mapsto_r v \vdash \text{wp bump} \{ w. \ulcorner w = 1 \urcorner * \ell \mapsto_r v \},$$

$$\text{wp bump_keep} \{ v. \ulcorner v = 0 \urcorner \}.$$

The closed run from σ_0 returns 0 (`bump_adequate`).

What is new. The struct lives in a stack slot, not on the raw heap. The field write is `wp_stack_field`: the slot changes from $\langle 0 \rangle$ to $\langle 1 \rangle$, and the read copies the component out with `wp_struct_load`. No raw address is allocated by `bump` itself.

The first theorem is the frame. Its hypothesis is an arbitrary raw cell, and the postcondition returns that same cell. The proof of the stack update never mentions it, so it cannot have been changed. The closed program makes the same point operationally: the fresh raw word is 0 before `bump` and 0 after, which is what the final load returns. The field update’s result 1 is established on the way and then discarded by sequencing, because the observation we keep is the unchanged raw word.

7.3 One raw cell: `cell.v`

Program.

```
// Generated by theories/examples/emit_cj.v from the CJR core terms.
```

```

foreign func malloc(size: UIntNative): CPointer<Unit>
foreign func free(ptr: CPointer<Unit>): Unit

func cell(): Unit {
    let p = unsafe { CPointer<Int64>(malloc(UIntNative(8 * 1))) }
    unsafe { p.write(7) }
    unsafe { p.read() }
    unsafe { free(CPointer<Unit>(p)) }
}

```

```

main(): Unit {
  cell()
}

```

Theorem.

$$\text{wp cell } \{ v. \ulcorner v = () \urcorner \}.$$

From σ_0 , the program returns unit and does not get stuck.

What is new. This is the first use of $\mapsto_r$ and $\mapsto_b$. Allocation produces $\ell \mapsto_r 0$ and $\ell \mapsto_b 1$. The store replaces the points-to fact by $\ell \mapsto_r 7$. The load reads 7 and returns the same fact. The program then discards that 7 by sequencing and frees the block. Free consumes both facts and returns unit.

The pure specification does not say “the loaded value was 7”; that fact is internal to the proof. What adequacy reports is that the whole expression, whose value is the value of **free**, returns unit without a stuck load, store, or double free. After **free**, no points-to fact remains, so a further load could not be verified.

7.4 Swap and aliasing: swap.v

Program.

```

// Generated by theories/examples/emit_cj.v from the CJR core terms.

foreign func malloc(size: UIntNative): CPointer<Unit>
foreign func free(ptr: CPointer<Unit>): Unit

func swap(l1: CPointer<Int64>, l2: CPointer<Int64>): Unit {
  let t = unsafe { l1.read() }
  unsafe { l1.write(unsafe { l2.read() }) }
  unsafe { l2.write(t) }
}

main(): Unit {
  let l1 = unsafe { CPointer<Int64>(malloc(UIntNative(8 * 1))) }
  let l2 = unsafe { CPointer<Int64>(malloc(UIntNative(8 * 1))) }
  unsafe { l1.write(1) }
  unsafe { l2.write(2) }
  swap(l1, l2)
}

```

Theorems.

$$\begin{aligned} \ell_1 \mapsto_r v_1 * \ell_2 \mapsto_r v_2 &\vdash \text{wp swap}(\ell_1, \ell_2) \{ _ . \ell_1 \mapsto_r v_2 * \ell_2 \mapsto_r v_1 \}, \\ \ell \mapsto_r v_1 * \ell \mapsto_r v_2 &\vdash \perp. \end{aligned}$$

What is new. The postcondition needs both cells. The proof loads ℓ_2 while still owning ℓ_1 , stores into ℓ_1 , then stores the saved value into ℓ_2 . Each rule mentions only the cell it touches; the other fact sits in the context and is framed.

The second lemma is `raw_mapsto_exclusive`, re-exported as `swap_aliased`. It is not a second proof of the same swap. It says that the swap specification cannot be instantiated at one address twice: the hypothesis of two points-to facts at that address is absurd. An overlapping specification would be a different theorem—for example a single cell updated twice—and it is not claimed here.

7.5 Indexed blocks: `sum_block.v`

Program.

```
// Generated by theories/examples/emit_cj.v from the CJR core terms.

foreign func malloc(size: UIntNative): CPointer<Unit>
foreign func free(ptr: CPointer<Unit>): Unit

func sumAt(p: CPointer<Int64>): Int64 {
  var i = 0
  var s = 0
  while (i <= 0) {
    s = s + unsafe { (p + i).read() }
    i = i + 1
  }
  s
}

main(): Unit {
  let p = unsafe { CPointer<Int64>(malloc(UIntNative(8 * 1))) }
  unsafe { p.write(4) }
  sumAt(p)
}
```

Theorem.

$$p \mapsto_r 4 * p \mapsto_b 1 \vdash \text{wpsum_at}(p) \{ v. \ulcorner v = 4 \urcorner * p \mapsto_r 4 * p \mapsto_b 1 \}.$$

What is new. The address that is loaded is not the base written in the source; it is the value of `Offset`, that is $p + i$. On the only iteration, i is 0, and $p + 0 = p$, so the load is justified by the points-to fact at p . The addition itself is the pure offset rule; it does not consume the points-to fact. The loop then increments i to 1, the condition $i \leq 0$ fails, and the sum 4 is returned.

The block-length fact is never used by the load, and it is still there at the end. That is the split between “I may read this word” and “I may free this block”. The example does not free. A load of $p + 1$ would not be provable from these hypotheses: there is no points-to fact at $p + 1$, and a one-word block does not contain that word.

The loop invariant, in the ordinary sense, is the assertion carried across the unfolding: i and s are stack cells, s holds the sum of the prefix already visited, and the raw word is unchanged. For a one-iteration loop the prefix is either empty or the single element 4.

7.6 Final classes: `class_upd.v`

Program.

```
// Generated by theories/examples/emit_cj.v from the CJR core terms.

foreign func malloc(size: UIntNative): CPointer<Unit>
```

```

foreign func free(ptr: CPointer<Unit>): Unit

class Box {
  public var value: Int64
  public init(value: Int64) {
    this.value = value
  }
}

func classBump(): Int64 {
  let r = unsafe { CPointer<Int64>(malloc(UIntNative(8 * 1))) }
  let o = Box(0)
  o.value = 5
  o.value
  unsafe { r.read() }
}

main(): Unit {
  classBump()
}

```

Theorem.

$$\text{wp class_bump } \{ v. \ulcorner v = 0 \urcorner \}.$$

From σ_0 , the program returns 0 and does not get stuck.

What is new. `new` allocates on the object map, not the raw map. The client receives $o \mapsto_o [0]0$. The field store changes it to $o \mapsto_o [0]5$, and the field load reads 5. The raw word allocated first is 0, is never stored, and is what the program returns.

So the closed observation is again the framed cell: object update does not change the raw heap. The proof makes the separation concrete. After `wp_alloc_one`, the raw points-to fact stays in the context through `wp_new`, `wp_field_store`, and `wp_field_load`. None of those lemmas takes a raw fact as a hypothesis. The final `wp_load` is what consumes it.

7.7 Framing across a call: `frame_call.v`

Program.

```

// Generated by theories/examples/emit_cj.v from the CJR core terms.

foreign func malloc(size: UIntNative): CPointer<Unit>
foreign func free(ptr: CPointer<Unit>): Unit

func write(x: CPointer<Int64>): Unit {
  unsafe { x.write(9) }
}

func frameCall(p: CPointer<Int64>, q: CPointer<Int64>): Int64 {
  write(p)
  unsafe { q.read() }
}

main(): Unit {

```

```

let p = unsafe { CPointer<Int64>(malloc(UIntNative(8 * 1))) }
let q = unsafe { CPointer<Int64>(malloc(UIntNative(8 * 1))) }
unsafe { q.write(3) }
frameCall(p, q)
}

```

Theorem.

$$p \mapsto_r v * q \mapsto_r 3 \vdash \text{wp write_call}(p, q) \{ w. \ulcorner w = 3 \urcorner * p \mapsto_r 9 * q \mapsto_r 3 \}.$$

What is new. The callee is a closure. `wp_rec` makes the closure, `wp_app` substitutes the pointer p for x , and `wp_store` needs only $p \mapsto_r v$. The fact $q \mapsto_r 3$ is not in the callee's precondition. It is framed across the call, then used by the load, which returns 3.

This is the frame rule applied at a function boundary. Nothing about `App` is special: substitution puts a value in the body, and the body is an ordinary store. The guarantee that q is unchanged is simply the guarantee that the store's proof did not receive q .

7.8 A hand-written vtable: `vtable.v`

Program.

```
// Generated by theories/examples/emit_cj.v from the CJR core terms.
```

```
foreign func malloc(size: UIntNative): CPointer<Unit>
foreign func free(ptr: CPointer<Unit>): Unit
```

```
struct VTable {
  public var inc: (Counter) -> Unit
  public init(inc: (Counter) -> Unit) {
    this.inc = inc
  }
}
```

```
class Counter {
  public var value: Int64
  public init(value: Int64) {
    this.value = value
  }
}
```

```
func inc(self: Counter): Unit {
  self.value = self.value + 1
}
```

```
func run(): Int64 {
  let box = VTable(inc)
  let o = Counter(0)
  box.inc(o)
  o.value
}
```

```
main(): Unit {
  run()
}
```

}

Theorem.

$$\text{wp vtable_prog} \{ v. \ulcorner v = 1 \urcorner \}.$$

From σ_0 , the program returns 1 and does not get stuck.

What is new. The function is a value, the struct is a tuple holding that value, and the call is an ordinary application of a component that was loaded out of the tuple. There is no method table in the machine and no dynamic dispatch rule. A “method” is a function whose first argument is the object, and a “vtable” is a struct the programmer assembled.

The proof follows that reading. First, the closure is formed by `wp_rec`. Second, the struct zipper evaluates the closure and becomes $\langle inc \rangle$ by the pure struct rules. `new` yields $o \mapsto_o [0]0$. `wp_struct_load` copies the closure out; this does not touch the object. `wp_app` substitutes the object reference into the body. The body loads field 0, adds one, and stores 1. Finally, the trailing field load reads that 1, which is the value of the whole program.

All three regions of the story meet: a stack-free unboxed struct of code, a code pointer, and an object field. The raw heap is not used. The earlier examples already showed that leaving it unused is safe.

8 A linked list

The file `theories/examples/list.v` is a self-contained implementation of a singly linked list, together with the equations that say the implementation really is a list. It is larger than the earlier examples because the specification is not a single pure result: it is a predicate on the object heap, and the programs are functions that allocate and read nodes.

8.1 Why a node is an object with a boolean tag

A textbook list cell is a pair of a head and a pointer to the rest, and the empty list is the null pointer. CJR has null, and loading through null is stuck, which is the right machine behaviour. What it does not have is an equality test on pointers or on object references. `EqOp` compares integers only. A program therefore cannot branch on “is this pointer null?”.

The sticking point is resolved by using a non-`open` class with three fields, and a boolean the language can already branch on.

```

Definition nil_body : expr :=
  Let (Some "n")
    (New [LitV (LitBool false); LitV (LitInt 0); LitV LitUnit] [])
    (Seq (FieldStore (Var "n") 2 (Var "n")) (Var "n")).

Definition cons_body : expr :=
  Let (Some "x") (StructLoad (Var "a") 0)
    (Let (Some "xs") (StructLoad (Var "a") 1)
      (New [] [Val (LitV (LitBool true)); Var "x"; Var "xs"])).

```

Field 0 is the tag: `false` means empty, `true` means cons. Field 1 is the head integer, which is 0 and unused when the tag is false. Field 2 is the tail. On an empty node it is that same node; on a cons node it is the value that represents the rest of the list, itself either an empty node or another

cons node. The allocation writes unit into the field and then stores the new object there before `nil` returns. The points-to fact owns the field. The value in the field is a pointer to the object, so the fact does not own the object a second time, and the predicate does not unfold. Objects are not freed, so a cons cell stays in the heap after its tail has been read. That matches the language: there is a new rule and no rule that deletes an object.

In Cangjie the operations are ordinary functions. Core application takes one argument, so `cons(x, xs)` lowers to a call on the struct of those two arguments. The other functions take the node.

```
// Generated by theories/examples/emit_cj.v from the CJR core terms.

foreign func malloc(size: UIntNative): CPointer<Unit>
foreign func free(ptr: CPointer<Unit>): Unit

class Node {
  public var isCons: Bool
  public var value: Int64
  var next_raw: ?Node
  public mut prop next: Node {
    get() {
      match (this.next_raw) {
        case Some(n) => n
        case None => this
      }
    }
    set(n) { this.next_raw = n }
  }
  public init(isCons: Bool, value: Int64, next: ?Node) {
    this.isCons = isCons
    this.value = value
    this.next_raw = next
  }
}

func nil(): Node {
  let n = Node(false, 0, None)
  n.next = n
  n
}

func cons(x: Int64, xs: Node): Node {
  Node(true, x, xs)
}

func head(n: Node): Int64 {
  n.value
}

func tail(n: Node): Node {
  n.next
}

func empty(n: Node): Bool {
```

```

    if (n.isCons) { false } else { true }
  }

main(): Unit {
  let e = nil()
  let l = cons(1, e)
  let h = head(l)
  let t = tail(l)
  empty(t)
}

```

A Cangjie constructor cannot receive the object it is building, so the printed class takes an optional tail and stores it in a raw field. Unit, the placeholder written by the allocation, is printed as **None**. The property **next** reads that raw field back as a **Node**: the stored node when it is present, and the node itself when it is empty. After **nil** returns, the raw tail has been set to the node. **empty** does not read **next**. The functions above are the Cangjie surface; **cons** of two arguments lowers to the unary core call on the struct $\langle x, xs \rangle$.

empty returns the negation of the tag. An empty node is empty; a cons node is not. The conditional is the only test. It does not inspect the tail pointer.

8.2 The representation predicate

On paper, a mathematical list xs is related to a value p by $\text{isList}(xs, p)$.

Definition 9 (List representation). $\text{isList}([], p)$ holds when $p = \text{obj}(o)$ and

$$o \mapsto_o [0]\text{false} * o \mapsto_o [1]0 * o \mapsto_o [2]\text{obj}(o).$$

$\text{isList}(x :: xs, p)$ holds when $p = \text{obj}(o)$ and there is a tail value q with

$$o \mapsto_o [0]\text{true} * o \mapsto_o [1]x * o \mapsto_o [2]q * \text{isList}(xs, q).$$

The Coq predicate is the same recursion, as a separation-logic assertion.

```

Fixpoint is_list (xs : list Z) (p : val) : iProp S :=
  match xs with
  | [] => exists o, pure (p = LitV (LitObj o)) *
    ObjId o |->[0] LitV (LitBool false) *
    ObjId o |->[1] LitV (LitInt 0) *
    ObjId o |->[2] LitV (LitObj o)
  | x :: xs' => exists o nxt, pure (p = LitV (LitObj o)) *
    ObjId o |->[0] LitV (LitBool true) *
    ObjId o |->[1] LitV (LitInt x) *
    ObjId o |->[2] nxt *
    is_list xs' nxt
  end.

```

What the separating conjunction is doing. The three fields of one node are three separate facts, so a proof may load the tag and still own the head and the tail. The recursive fact $\text{isList}(xs, q)$ owns every node of the tail and nothing of the cons cell. Those two regions are disjoint. That is why “the tail is the original list” can be stated by returning the original pointer together with $\text{isList}(xs, p)$, while the cons cell’s own three fields sit beside it rather than inside it.

A value that is not an object reference does not satisfy `isList`. The lemma `is_list_obj` records the useful direction: if the predicate holds, the value is `obj(o)` for some o . The proofs rewrite with that equality before they substitute, so an abstract pointer is a literal object reference by the time it is placed in a struct. Substitution then computes.

8.3 What each operation establishes

Empty list.

$$\text{wp nil}() \{ v. \text{isList}([], v) \}.$$

`wp_new` of the three initialising values returns three points-to facts, with unit in the tail. `wp_field_store` then writes the new object into that field, which is the empty clause. Nothing else is allocated. This is `nil_spec`.

Cons.

$$\text{isList}(xs, p) \vdash \text{wp cons } \langle x, p \rangle \left\{ v. \exists o. v = \text{obj}(o) * o \mapsto_o [0] \text{true} * o \mapsto_o [1] x * o \mapsto_o [2] p * \text{isList}(xs, p) \right\}.$$

The argument struct is unboxed. `wp_struct_load` copies x and then p out of it; those loads do not touch the heap. The body then allocates one object. The third field is the original pointer p , not a copy of the tail's nodes. The fact `isList(xs, p)` is framed across allocation: `wp_new` does not ask for it, so it cannot be changed. The postcondition is the unfolding of `isList(x :: xs, v)` with the tail pointer pinned to the pointer that was passed in. Pinning it is what makes the later inverse law an equality of values, not only an equality of mathematical lists up to some existential. This is `cons_spec`.

Head.

$$\text{isList}(x :: xs, p) \vdash \text{wp head } p \{ v. \lceil v = x \rceil * \text{isList}(x :: xs, p) \}.$$

The proof opens the cons clause, loads field 1, and puts the same facts back together. The list predicate after the call is the list predicate before the call. Head is a pure observation. This is `head_spec`.

Tail.

$$\begin{aligned} &\text{isList}(x :: xs, p) \vdash \\ &\text{wp tail } p \left\{ v. \text{isList}(xs, v) * \exists o. p = \text{obj}(o) * o \mapsto_o [0] \text{true} * o \mapsto_o [1] x * o \mapsto_o [2] v \right\}. \end{aligned}$$

Loading field 2 returns the tail pointer and gives the points-to fact back. The recursive clause of `isList` already owned `isList(xs, v)` separately from the cons cell, so the postcondition splits the list into the cell that was examined and the list that remains. It does not duplicate nodes. This is `tail_spec`.

Emptiness.

$$\begin{aligned} &\text{isList}([], p) \vdash \text{wp empty } p \{ v. \lceil v = \text{true} \rceil * \text{isList}([], p) \}, \\ &\text{isList}(x :: xs, p) \vdash \text{wp empty } p \{ v. \lceil v = \text{false} \rceil * \text{isList}(x :: xs, p) \}. \end{aligned}$$

Both proofs load the tag and take one branch of `if`. The other branch is dead in that specification: a proof of the empty law never has to show anything about the cons branch, and conversely. The predicate is restored, so emptiness is an observation rather than a destructive read. These are `empty_nil` and `empty_cons`.

8.4 Adding an element and removing it

The program that the inverse law talks about is the composition of the two functions, not a fused implementation.

```
Definition cons_then_tail (x : Z) (p : val) : expr :=
  Let (Some "node") (call_cons x p)
  (App (Rec None (Some "n") tail_body) (Var "node")).
```

Theorem 10 (Cons then tail).

$$\text{isList}(xs, p) \vdash \text{wp}(\text{tail}(\text{cons}(x, p))) \{ v. \ulcorner v = p \urcorner * \text{isList}(xs, v) \}.$$

The Coq statement is `cons_then_tail_spec`. Read it as an equation of lists. The value returned by removing the element just added is the original pointer, and that pointer still represents the original mathematical list.

The proof follows the program in order.

1. `cons_spec` allocates the node and frames $\text{isList}(xs, p)$. Its postcondition says field 2 of the new object is exactly p , and the original list predicate is still owned.
2. The `let` substitutes that object for `node`.
3. `tail` is the function above. It loads field 2. The load rule returns the value that the points-to fact describes, which is p , and it returns the points-to fact as well.
4. The `cons` cell's three fields are no longer needed for the equation. They are discarded. Iris assertions are affine, so ownership of leftover cells may be dropped. The object remains allocated in the machine; the specification simply stops mentioning it. What remains is $\ulcorner v = p \urcorner$ and $\text{isList}(xs, p)$.

Two companion readings are already in the lemmas above, and they are the other usual equations. Head after `cons` returns x and still owns $\text{isList}(x :: xs, \cdot)$. Emptiness distinguishes `[]` from $x :: xs$. Together they say that the tag, the head field, and the tail field agree with the mathematical constructors, and that reading them does not damage the list those constructors describe.

What this development does not prove is induction over an arbitrary list operation such as `append` or `reverse`. The predicate is defined so that those proofs are the same shape: open one `cons` cell, recurse on the tail fact, and frame the cell. The equations above are the base that such a proof would use.

9 Arrays

The file `theories/examples/array.v` is a library, in the same sense as the list: the language has no array type, and the library builds one from a final class and a raw block. An array of n integers is an object with two fields and a contiguous block of n words. Field 0 holds n . Field 1 holds the base pointer of the block. Word i of the block holds element i . The index is a computed integer, so the elements cannot be object fields: a field index in the core is a static natural number, whereas `Offset` and `Load` take a value.

`Alloc` of zero words is stuck, so `make` asks for a positive length. An index outside $0, \dots, n - 1$ is likewise stuck: the specification owns only the words of the block, and a load with no points-to fact does not reduce.

9.1 The library

In Cangjie the value is a non-open class. Allocation of n words is `malloc` of n times the size of `Int64`; the core step is `Alloc` of the word count n , which returns a pointer to n zero words. `get` and `set` take the array and the index, and `set` also takes the new element. Core application is unary, so those calls lower to one struct argument, as `cons` did.

```
// Generated by theories/examples/emit_cj.v from the CJR core terms.

foreign func malloc(size: UIntNative): CPointer<Unit>
foreign func free(ptr: CPointer<Unit>): Unit

class Array {
  public var len: Int64
  public var data: CPointer<Int64>
  public init(len: Int64, data: CPointer<Int64>) {
    this.len = len
    this.data = data
  }
}

func make(n: Int64): Array {
  let p = unsafe { CPointer<Int64>(malloc(UIntNative(8 * n))) }
  Array(n, p)
}

func length(a: Array): Int64 {
  a.len
}

func get(arr: Array, i: Int64): Int64 {
  let p = arr.data
  unsafe { (p + i).read() }
}

func set(arr: Array, i: Int64, x: Int64): Unit {
  let p = arr.data
  unsafe { (p + i).write(x) }
}

main(): Unit {
  let a = make(4)
  set(a, 0, 7)
  let x = get(a, 0)
  length(a)
}
```

The core terms are the same operations with the struct lowering made explicit.

```
Definition make_body : expr :=
  Let (Some "p") (Alloc (Var "n"))
  (New [] [Var "n"; Var "p"]).

Definition get_body : expr :=
```

```

Let (Some "arr") (StructLoad (Var "args") 0)
  (Let (Some "i") (StructLoad (Var "args") 1)
    (Let (Some "p") (FieldLoad (Var "arr") 1)
      (Load (Offset (Var "p") (Var "i"))))).

Definition set_body : expr :=
  Let (Some "arr") (StructLoad (Var "args") 0)
    (Let (Some "i") (StructLoad (Var "args") 1)
      (Let (Some "x") (StructLoad (Var "args") 2)
        (Let (Some "p") (FieldLoad (Var "arr") 1)
          (Store (Offset (Var "p") (Var "i")) (Var "x"))))).

```

`length` is a field load of index 0.

9.2 The representation

Definition 11 (Array representation). $\text{isArray}(xs, a)$ holds when $a = \text{obj}(o)$, the base pointer is p , and

$$o \mapsto_o [0]|xs| * o \mapsto_o [1]\text{ptr}(p) * p \mapsto_b |xs| * \bigstar_{i < |xs|} (p + i) \mapsto_r xs_i.$$

The block fact is the right to treat $[p, p + |xs|)$ as one allocation. The separating conjunction of the words is what lets a proof load or store one index and still own the others. Two different indices are two different addresses, so the facts do not overlap.

The Coq predicate is that assertion. `array_cells` is the big separating conjunction, indexed by position in xs .

```

Definition array_cells (base : Z) (xs : list Z) : iProp S :=
  (* separating conjunction over i |-> xs_i at address base+i *)
  [ * list] i |-> x in xs,
  RawId (base + Z.of_nat i) |->r LitV (LitInt x).

Definition is_array (xs : list Z) (a : val) : iProp S :=
  exists o base, pure (a = LitV (LitObj o)) *
  ObjId o |->o[0] LitV (LitInt (Z.of_nat (length xs))) *
  ObjId o |->o[1] LitV (LitPtr base) *
  RawId base |->b (length xs) *
  array_cells base xs.

```

9.3 Allocation of n words

The one-word rule is not enough for `make`. The machine already steps `Alloc` of any positive word count: it zeroes that many words, records the length at the base, and advances the counter by the count. `wp_alloc` is the matching logic rule. From $0 < n$,

$$\text{wp_alloc}(n) \left\{ v. \exists \ell. v = \text{ptr}(\ell) * \ell \mapsto_b n * \bigstar_{i < n} (\ell + i) \mapsto_r 0 \right\}.$$

The proof is the one-word proof with a finite map of cells instead of a singleton. `init_cells` builds that map, `wf_alloc` shows the resulting machine is well formed, and the ghost-map update hands back one points-to fact per word.

9.4 What each operation establishes

Make. For $0 < n$,

$$\text{wp make}(n) \{ a. \text{isArray}(\text{replicate}(n, 0), a) \}.$$

Allocation produces the zero words and the block fact. The following **new** stores n and the pointer in the object. This is **make_spec**.

Length.

$$\text{isArray}(xs, a) \vdash \text{wp length}(a) \{ v. \ulcorner v = |xs| \urcorner * \text{isArray}(xs, a) \}.$$

The proof loads field 0 and returns the same predicate. Length is an observation. This is **length_spec**.

Get. If $xs_i = x$, writing i for the mathematical index,

$$\text{isArray}(xs, a) \vdash \text{wp get}(a, i) \{ v. \ulcorner v = x \urcorner * \text{isArray}(xs, a) \}.$$

The body reads the pointer from field 1, adds i , and loads that word. The lookup lemma for a big separating conjunction lends out the one cell and takes it back, so the other cells are not mentioned by the load rule and stay in the predicate. This is **get_spec**.

Set. If index i is in range,

$$\text{isArray}(xs, a) \vdash \text{wp set}(a, i, x) \{ v. \ulcorner v = () \urcorner * \text{isArray}(xs[i \mapsto x], a) \}.$$

The store updates that one word. The length stored in the object and the block length are unchanged, because replacing an element does not change $|xs|$. This is **set_spec**.

9.5 The array equations

The programs are the compositions of the functions above. Each one is a **let** of one call followed by the next, not a fused body.

Theorem 12 (Read after write). *If index i is in range, then*

$$\text{isArray}(xs, a) \vdash \text{wp}(\text{get}(\text{set}(a, i, x), i)) \{ v. \ulcorner v = x \urcorner * \text{isArray}(xs[i \mapsto x], a) \}.$$

If j is also in range and $i \neq j$, then

$$\text{isArray}(xs, a) \vdash \text{wp}(\text{get}(\text{set}(a, i, x), j)) \{ v. \ulcorner v = xs_j \urcorner * \text{isArray}(xs[i \mapsto x], a) \}.$$

These are **set_then_get_same** and **set_then_get_other**. The first is **set_spec** followed by **get_spec** at the updated list, whose lookup at i is x . The second uses the same **set_spec**, then **get_spec** at j . The lookup of an updated list at a different index is the old element. The separating conjunction is why the store at i does not have a points-to fact for j among its hypotheses: that fact is framed, so it cannot change.

Theorem 13 (Length is invariant). *If index i is in range, then*

$$\text{isArray}(xs, a) \vdash \text{wp}(\text{length}(\text{set}(a, i, x))) \{ v. \ulcorner v = |xs| \urcorner * \text{isArray}(xs[i \mapsto x], a) \}.$$

For $0 < n$ and $i < n$,

$$\begin{aligned} & \text{wp}(\text{length}(\text{make}(n))) \{ v. \exists a. \ulcorner v = n \urcorner * \text{isArray}(\text{replicate}(n, 0), a) \}, \\ & \text{wp}(\text{get}(\text{make}(n), i)) \{ v. \exists a. \ulcorner v = 0 \urcorner * \text{isArray}(\text{replicate}(n, 0), a) \}. \end{aligned}$$

The length equation is `set_then_length_spec`: after `set`, `length` reads the field that still holds $|xs|$. The two make equations are `make_then_length_spec` and `make_then_get_spec`. A fresh array is a vector of zeroes of the requested length, and reading it does not change that fact.

What this development does not prove is an arbitrary loop over the array, such as a sum of every cell. The cells are already a separating conjunction indexed by the mathematical list, which is the induction hypothesis such a loop would use. The equations above are the base: read and write agree with list update, and they do not disturb the length or the other cells.

10 Array lists

The file `theories/examples/array_list.v` lowers the resizable-buffer core of Cangjie's `ArrayList`: a default constructor, the live length, the capacity, emptiness, reading and writing an index, append, removal at an index, growth, and the version counter. The source class continues past that core, with an iterator, the `Collection` interface, ranges, higher-order methods, and an embedded sort. Those stay out. The core is monomorphic in `Int64`, and it uses the array library already proved: a final object, a raw block, and a word-by-word copy.

`Alloc` of zero words is stuck, so the only constructor is the default one, at capacity 16. There is no shift, so growth doubles by adding the capacity to itself. There are no exceptions. An index outside $[0, size)$ lowers to a load of a non-pointer, which has no step, and every specification takes the arm that is in range. `set` leaves the version alone. `add` and `remove` increment it. Growth replaces the buffer and drops the old block and the old array object. The source relies on collection for that drop. This language has a one-word free and no collection, so the old facts are not returned to the allocator. Iris is affine, and the proof simply does not use them again.

10.1 The library

In Cangjie the value is a non-`open` class. Field 0 is the buffer, itself the array of the previous section. Field 1 is the live length. Field 2 is the version. The buffer's own field 0 is the capacity, and its field 1 is the base pointer of a block of that many words. The live prefix of the block is the mathematical list. The tail is zero, which is what `Alloc` writes and what removal writes back.

```
// Generated by theories/examples/emit_cj.v from the CJR core terms.

foreign func malloc(size: UIntNative): CPointer<Unit>
foreign func free(ptr: CPointer<Unit>): Unit

class Array {
  public var len: Int64
  public var data: CPointer<Int64>
  public init(len: Int64, data: CPointer<Int64>) {
    this.len = len
    this.data = data
  }
}

class ArrayList {
  public var myData: Array
  public var mySize: Int64
  public var myVersion: Int64
  public init(myData: Array, mySize: Int64, myVersion: Int64) {
```

```

        this.myData = myData
        this.mySize = mySize
        this.myVersion = myVersion
    }

    public func size(): Int64 {
        this.mySize
    }

    public func capacity(): Int64 {
        let buf = this.myData
        buf.len
    }

    public func version(): Int64 {
        this.myVersion
    }

    public func isEmpty(): Bool {
        let n = this.mySize
        n == 0
    }

    public func get(i: Int64): Int64 {
        let n = this.mySize
        if (n <= i) {
            unsafe { CPointer<Int64>().read() }
        } else {
            if (i <= -1) {
                unsafe { CPointer<Int64>().read() }
            } else {
                let buf = this.myData
                let p = buf.data
                unsafe { (p + i).read() }
            }
        }
    }

    public func set(i: Int64, x: Int64): Unit {
        let n = this.mySize
        if (n <= i) {
            unsafe { CPointer<Int64>().read() }
        } else {
            if (i <= -1) {
                unsafe { CPointer<Int64>().read() }
            } else {
                let buf = this.myData
                let p = buf.data
                unsafe { (p + i).write(x) }
            }
        }
    }

    public func grow(min: Int64): Unit {

```

```

    let buf = this.myData
    let pcap = buf.len
    let doubled = pcap + pcap
    let newCap = if (min <= doubled) { doubled } else { min }
    let src = buf.data
    let len = this.mySize
    let soff = 0
    let doff = 0
    let dst = unsafe { CPointer<Int64>(malloc(UIntNative(8 * newCap))) }
    let nb = Array(newCap, dst)
    var i = 0
    while ((i + 1) <= len) {
        unsafe { (dst + (i + doff)).write(unsafe { (src + (i + soff)).read() }) }
        i = i + 1
    }
    this.myData = nb
}

public func add(x: Int64): Unit {
    let n = this.mySize
    let buf = this.myData
    let pcap = buf.len
    let min = n + 1
    if (pcap <= n) {
        this.grow(min)
    }
    let buf2 = this.myData
    let p = buf2.data
    unsafe { (p + n).write(x) }
    let n1 = n + 1
    this.mySize = n1
    let ver = this.myVersion
    this.myVersion = ver + 1
}

public func remove(at: Int64): Int64 {
    let n = this.mySize
    if (n <= at) {
        unsafe { CPointer<Int64>().read() }
    } else {
        if (at <= -1) {
            unsafe { CPointer<Int64>().read() }
        } else {
            let buf = this.myData
            let p = buf.data
            let removed = unsafe { (p + at).read() }
            let src = p
            let dst = p
            let soff = at + 1
            let doff = at
            let len = (n - at) - 1
            var i = 0
            while ((i + 1) <= len) {
                unsafe { (dst + (i + doff)).write(unsafe { (src + (i + soff)).read() }) }
            }
        }
    }
}

```

```

    }) }

        i = i + 1
    }
    let last = n - 1
    unsafe { (p + last).write(0) }
    this.mySize = last
    let ver = this.myVersion
    this.myVersion = ver + 1
    removed
  }
}
}

func initArrayList(): ArrayList {
  let p = unsafe { CPointer<Int64>(malloc(UIntNative(8 * 16))) }
  let buf = Array(16, p)
  ArrayList(buf, 0, 0)
}

main(): Unit {
  let a = initArrayList()
  a.add(1)
  a.add(2)
  let r = a.remove(0)
  a.size()
}

```

Core application is unary, so each method lowers to a closure applied to one struct of arguments, as `get` and `set` did for arrays. The pure model of the three mutating operations is ordinary list algebra.

```

Definition al_set (xs : list Z) (i : nat) (x : Z) : list Z := <[i := x]> xs.
Definition al_add (xs : list Z) (x : Z) : list Z := xs ++ [x].
Definition al_remove (xs : list Z) (i : nat) : list Z :=
  take i xs ++ drop (S i) xs.

```

Appending puts x at index $|xs|$ and leaves every earlier index unchanged. Updating index i reads back x there and leaves every other index unchanged. Removing index i shortens the list by one and shifts the suffix down.

10.2 The representation

Two objects. The inner one is the buffer. The outer one is the list.

Definition 14 (Array-list representation). Write $\text{slots}(xs, c)$ for xs followed by $c - |xs|$ zeroes, which is defined only for $|xs| \leq c$. $\text{isBuf}(\text{slots}, b)$ is the array predicate of the previous section with the whole capacity in place of the live prefix: field 0 holds $|\text{slots}|$, field 1 holds the base pointer, and the block owns every word of slots . $\text{isArrayList}(xs, c, v, a)$ holds when $0 < c$, $|xs| \leq c$, $a = \text{obj}(o)$, and

$$o \mapsto_o [0]b * o \mapsto_o [1]|xs| * o \mapsto_o [2]v * \text{isBuf}(\text{slots}(xs, c), b).$$

The Coq predicates are `slots_of`, `is_buf`, and `is_arraylist`. The cells are `array_cells`, reused from the array library, so a proof of a single load or store is the same lookup lemma as `get` and `set`

there. The zero tail is part of the owned block. Appending into spare capacity overwrites the first zero. Removing the last live word writes a zero back, so the tail stays zero.

10.3 Copy and growth

Copy is one loop. It binds an index at 0 and, while that index plus one is at most the length, loads one word and stores it one step further along. The loop is `copy_vars`. Two invariants share that body.

The first is a copy between two blocks, used by growth. Source and destination do not overlap, so each iteration may read the source and write the destination in either order. `copy_disjoint` says that after `len` steps the destination holds the source chunk at the destination offset, and the source is unchanged.

The second is a forward shift on one block, used by removal. The source offset is the destination offset plus one, so each word is read before the next store overwrites it. `copy_shift` says that the live prefix is the list with the suffix shifted down by one. A left shift of length zero, which is removal of the last element, is the loop condition failing at once.

Growth asks for a minimum capacity. The new capacity is $2c$ when the minimum is at most $2c$, and the minimum otherwise.

$$\text{growCap}(c, m) = \begin{cases} 2c & \text{if } m \leq 2c, \\ m & \text{otherwise.} \end{cases}$$

Both arms allocate a positive block, copy the live prefix into it, and store the new array object in field 0. The mathematical list and the version are unchanged. The old block remains allocated and is not mentioned by the postcondition. This is `grow_spec`. The Coq name of the capacity is `grow_cap`; doubling is `doubled`, which is addition because the core has no shift.

Append uses that rule only when the live length already equals the capacity. Then the minimum is the length plus one, which is at most $2c$ for $c > 0$, so the capacity doubles. When the length is strictly below the capacity, append writes into the zero tail and the capacity stays.

10.4 What each operation establishes

Init.

$$\text{wp init}() \{ a. \text{isArrayList}([], 16, 0, a) \}.$$

Allocation of 16 words produces the zero block. The inner object stores 16 and the pointer. The outer object stores that buffer, length 0, and version 0. This is `init_spec`. `default_capacity` is 16.

Size, capacity, emptiness, version. Each is a field load, or a load followed by a comparison with zero.

$$\begin{aligned} \text{isArrayList}(xs, c, v, a) &\vdash \text{wp size}(a) \{ r. \ulcorner r = |xs| \urcorner * \text{isArrayList}(xs, c, v, a) \}, \\ \text{isArrayList}(xs, c, v, a) &\vdash \text{wp capacity}(a) \{ r. \ulcorner r = c \urcorner * \text{isArrayList}(xs, c, v, a) \}, \\ \text{isArrayList}(xs, c, v, a) &\vdash \text{wp isEmpty}(a) \{ r. \ulcorner r = (|xs| = 0) \urcorner * \text{isArrayList}(xs, c, v, a) \}, \\ \text{isArrayList}(xs, c, v, a) &\vdash \text{wp version}(a) \{ r. \ulcorner r = v \urcorner * \text{isArrayList}(xs, c, v, a) \}. \end{aligned}$$

These are `size_spec`, `capacity_spec`, `is_empty_spec`, and `version_spec`. Capacity reads the buffer's length field, which `slots_of` keeps equal to c .

Get and set. If $xs_i = y$,

$$\begin{aligned} \text{isArrayList}(xs, c, v, a) &\vdash \text{wp get}(a, i) \{ r. \ulcorner r = y \urcorner * \text{isArrayList}(xs, c, v, a) \}, \\ \text{isArrayList}(xs, c, v, a) &\vdash \text{wp set}(a, i, x) \{ r. \ulcorner r = () \urcorner * \text{isArrayList}(xs[i \mapsto x], c, v, a) \}. \end{aligned}$$

Both go through the buffer pointer and the raw word at index i . The range test is two comparisons, c against the index and the index against -1 , and the specification is the success arm. **set** stores into the live prefix and does not touch field 2. These are **al_get_spec** and **al_set_spec**. The names are prefixed because the array library already exports **get_spec** and **set_spec**.

Add.

$$\text{isArrayList}(xs, c, v, a) \vdash \text{wp add}(a, x) \{ r. \ulcorner r = () \urcorner * \text{isArrayList}(xs ++ [x], \text{addCap}(c, |xs|), v + 1, a) \}.$$

$\text{addCap}(c, n)$ is c when $n < c$, and $\text{growCap}(c, n + 1)$ otherwise. The body writes x at the old length, stores the new length, and increments the version. When the buffer is already full it first calls **grow**. This is **add_spec**.

Remove. If $xs_j = y$,

$$\text{isArrayList}(xs, c, v, a) \vdash \text{wp remove}(a, j) \{ r. \ulcorner r = y \urcorner * \text{isArrayList}(\text{delete}(xs, j), c, v + 1, a) \}.$$

$\text{delete}(xs, j)$ is the prefix of length j followed by the suffix after j . The body loads y before the shift, copies the suffix down by one, writes zero at the old last index, stores the new length, and increments the version. The capacity is the same block. This is **remove_spec**.

10.5 The array-list equations

The programs are compositions of the calls above. Each one is a **let** of one call followed by the next, proved from the triples by framing, as **set_then_get** was for arrays.

Theorem 15 (A fresh list).

$$\begin{aligned} &\text{wp}(\text{size}(\text{init}())) \{ r. \exists a. \ulcorner r = 0 \urcorner * \text{isArrayList}([], 16, 0, a) \}, \\ &\text{wp}(\text{isEmpty}(\text{init}())) \{ r. \exists a. \ulcorner r = \text{true} \urcorner * \text{isArrayList}([], 16, 0, a) \}, \\ &\text{wp}(\text{capacity}(\text{init}())) \{ r. \exists a. \ulcorner r = 16 \urcorner * \text{isArrayList}([], 16, 0, a) \}, \\ &\text{wp}(\text{version}(\text{init}())) \{ r. \exists a. \ulcorner r = 0 \urcorner * \text{isArrayList}([], 16, 0, a) \}. \end{aligned}$$

Size and emptiness are **init_then_size_spec** and **init_then_empty_spec**. Capacity and version are **init_then_capacity_spec** and **init_then_version_spec**.

Theorem 16 (Append).

$$\begin{aligned} &\text{isArrayList}(xs, c, v, a) \vdash \text{wp}(\text{size}(\text{add}(a, x))) \\ &\quad \{ r. \ulcorner r = |xs| + 1 \urcorner * \text{isArrayList}(xs ++ [x], \text{addCap}(c, |xs|), v + 1, a) \}, \\ &\text{isArrayList}(xs, c, v, a) \vdash \text{wp}(\text{get}(\text{add}(a, x), |xs|)) \\ &\quad \{ r. \ulcorner r = x \urcorner * \text{isArrayList}(xs ++ [x], \text{addCap}(c, |xs|), v + 1, a) \}. \end{aligned}$$

If $i < |xs|$ and $xs_i = y$, the same append followed by **get** at i returns y . The capacity is c when $|xs| < c$, and $2c$ when $|xs| = c$. The version after append is $v + 1$.

Size and the new-index lookup are `add_then_size_spec` and `add_then_get_new_spec`. An older index is `add_then_get_old_spec`. Spare capacity is `add_then_capacity_room_spec`, a full buffer is `add_then_capacity_full_spec`, and the version is `add_then_version_spec`. The lookup at the new index is the lemma that an appended element sits at the old length. The lookup at an older index is the lemma that append does not change a shorter prefix, including across the growth that replaces the block.

Theorem 17 (Update). *If $xs_i = y$, then reading index i after writing x there returns x , and the version is still v . If j is also in range and $i \neq j$, reading j returns xs_j . The length after the update is still $|xs|$.*

The two lookups are `set_then_get_same_spec` and `set_then_get_other_spec`. Length and version are `set_then_size_spec` and `set_then_version_spec`. The version equation is the observation that `set` does not store field 2.

Theorem 18 (Append then remove).

$$\begin{aligned} \text{isArrayList}(xs, c, v, a) \vdash \text{wp}(\text{remove}(\text{add}(a, x), |xs|)) \\ \{ r. \ulcorner r = x \urcorner * \text{isArrayList}(xs, \text{addCap}(c, |xs|), v + 2, a) \}. \end{aligned}$$

Removing an in-range index and then reading the size returns $|xs| - 1$.

The round trip is `add_then_remove_last_spec`. Removal of the element just appended restores xs . The capacity is whatever append left behind, which may be $2c$, and the version has been incremented twice. The size equation is `remove_then_size_spec`.

What this development does not prove is the iterator, equality of two lists, or a bulk operation such as `filter`. The predicate already owns the live prefix and the zero tail as one separating conjunction, which is the induction hypothesis a further loop would use. The equations above are the base: length, lookup, update, append, and removal agree with the list algebra, growth only changes the capacity, and the version moves only on append and removal.

11 What is deliberately missing

We deliberately omit concurrency from the semantics, so there is no invariant-opening rule and no fork. Float literals exist, yet no float operator does. Allocation of a positive word count is the rule used by the array library. Freeing in the named logic rules remains the one-word rule. Objects are not collected. Pointer arithmetic is word-addressed. Null, a freed address, and an address with no points-to fact are stuck rather than given a defined error value.

The type grammar is not enforced. The adequacy theorem is the safety theorem for those programs whose specifications have been proved.

12 Reading list

The entries below are the sources this note relies on. The formal bibliography follows the same order.

The language. The Cangjie 1.0 documentation [1] is the surface syntax from which this fragment is cut: `func`, `let` and `var`, `if` and `while`, `struct`, non-open `class`, and `CPointer`. The performance-parity cut keeps unboxed structs, word-addressed raw blocks, and final classes, with generics, inheritance, and concurrency left as sugar that must lower into that cut.

Separation logic. Reynolds [2] is the logic of disjoint ownership of mutable structure, which is the reading of $\mapsto_s$, $\mapsto_r$, $\mapsto_b$, and $\mapsto_o$ in this development. O’Hearn, Reynolds, and Yang [3] is the local-reasoning paper: a specification mentions only the cells the command uses, and the frame rule supplies the rest. That is the justification of the framing steps in the examples and in `cons_spec`.

Iris. The POPL 2015 paper [4] introduces the resource algebra that the four ghost maps instantiate. The ESOP 2017 paper [5] is the one to read for the program logic used here: weakest preconditions, evaluation contexts, the bind rule, and the lifting of a deterministic head step. *Iris from the ground up* [6] is the self-contained account of the same logic, including adequacy. The Iris Proof Mode paper [7] explains the `iApply`, `iDestruct`, and `iFrame` style of the Coq proofs. The lecture notes [8] are the shortest path through all of that with exercises. HeapLang, in the Iris 4.4.0 Coq development [9], is the language whose `ectxi_language` instance this one copies; `ghost_map` is the library behind the points-to facts.

The proof assistant. The mechanization is checked with Coq 8.20.1 [10] and std++ 1.12.0 [11]. std++ supplies the finite maps, the list lemmas, and the `decide` tactic used for the pure arithmetic steps.

References

- [1] Cangjie documentation, version 1.0.0. <https://docs.cangjie-lang.cn/en/docs/1.0.0/>.
- [2] John C. Reynolds. Separation logic: A logic for shared mutable data structures. In *Proceedings of LICS*, pages 55–74. IEEE, 2002.
- [3] Peter W. O’Hearn, John C. Reynolds, and Hongseok Yang. Local reasoning about programs that alter data structures. In *Proceedings of CSL*, volume 2142 of *LNCS*, pages 1–19. Springer, 2001.
- [4] Ralf Jung, David Swasey, Filip Sieczkowski, Kasper Svendsen, Aaron Turon, Lars Birkedal, and Derek Dreyer. Iris: Monoids and invariants as an orthogonal basis for concurrent reasoning. In *Proceedings of POPL*, pages 637–650. ACM, 2015.
- [5] Robbert Krebbers, Ralf Jung, Aleš Bizjak, Jacques-Henri Jourdan, Derek Dreyer, and Lars Birkedal. The essence of higher-order concurrent separation logic. In *Proceedings of ESOP*, volume 10201 of *LNCS*, pages 696–723. Springer, 2017.
- [6] Ralf Jung, Robbert Krebbers, Jacques-Henri Jourdan, Aleš Bizjak, Lars Birkedal, and Derek Dreyer. Iris from the ground up: A modular foundation for higher-order concurrent separation logic. *Journal of Functional Programming*, 28:e20, 2018.
- [7] Robbert Krebbers, Amin Timany, and Lars Birkedal. Interactive proofs in higher-order concurrent separation logic. In *Proceedings of POPL*, pages 205–217. ACM, 2017.
- [8] Lars Birkedal and Aleš Bizjak. Lecture notes on Iris: Higher-order concurrent separation logic. <https://iris-project.org/pdfs/iris-lecture-notes.pdf>.
- [9] The Iris Project. The Iris Coq development, version 4.4.0, including HeapLang. <https://gitlab.mpi-sws.org/iris/iris>.

- [10] The Coq Development Team. The Coq proof assistant, version 8.20.1. <https://coq.inria.fr/>.
- [11] The Iris Project. coq-std++, version 1.12.0. <https://gitlab.mpi-sws.org/iris/stdpp>.